

① Q. Application Not Working in Production. How will you troubleshoot?

Interview Answer :

Whenever I receive an alert that the production application is down, I don't immediately restart the service. My first step is to understand the impact of the issue by checking whether all users are affected or only a specific module. I also verify whether the issue started after a recent deployment or infrastructure change.

Next, I check our monitoring dashboards such as CloudWatch, Grafana and Prometheus to identify any abnormal CPU, memory, pod restart, latency or error spikes. This gives me an initial understanding of whether the issue is application-related or infrastructure-related.

If the application is running on Kubernetes, I first verify the pod status using `kubectl get pods`. If I find `CrashLoopBackOff`, `ImagePullBackOff` or `OOMKilled` errors, I inspect the pod using `kubectl describe pod` and check the application logs using `kubectl logs` to identify the root cause.

If all pods are healthy, I verify the Kubernetes Service, Ingress, Load Balancer and Target Group health. I also confirm that the application is reachable from the network and DNS is resolving correctly.

Then I review the application logs for exceptions such as database connection failures, authentication issues, timeout errors or configuration problems. If required, I also verify dependencies like RDS, Redis, Kafka, external APIs or third-party services.

I always check the Jenkins deployment history and Git commits to see whether a recent release introduced the issue. If the production outage is caused by the latest deployment, I perform a rollback to the last stable version using Kubernetes rollout or the previous Jenkins build.

After restoring the application, I continue monitoring the service for some time to ensure stability. Finally, I prepare a Root Cause Analysis (RCA), document the findings, and implement preventive actions such as health checks, monitoring alerts or pipeline validations to avoid similar incidents in the future.

Troubleshooting Flow



- ① Check Monitoring Dashboards (CloudWatch / Grafana / Prometheus)
- ↓
- ② Check Recent Deployments / Changes
- ↓
- ③ Check Kubernetes Resources (Pods / Events / Logs)
- ↓
- ④ Check Service, Ingress, ALB & Target Group Health
- ↓
- ⑤ Check Infrastructure & Network (EC2 / SG / NACL / VPC / DNS)
- ↓
- ⑥ Check Dependencies (RDS / Redis / Kafka / APIs)
- ↓
- ⑦ Analyze Application Logs
- ↓
- ⑧ Rollback if required to last stable version
- ↓
- ⑨ Monitor & Validate
- ↓
- ⑩ Perform RCA & Prevent future

Important Commands



- `kubectl get pods -A` # Check pods
- `kubectl get events -A` # Check events
- `kubectl describe pod <pod>` # Pod details
- `kubectl logs <pod> -n <ns>` # Pod Logs
- `kubectl logs <pod> --previous` # Previous logs
- `kubectl rollout history deployment/<app>` # Rollout history
- `kubectl rollout undo deployment/<app>` # Rollback
- `aws elbv2 describe-target-health` # TG health
- `aws rds describe-db-instances` # RDS status
- `curl -I http://app-endpoint/health` # Health check



Key Responsibilities

- Monitor production applications using CloudWatch, Grafana and Prometheus.
- Troubleshoot Kubernetes (EKS) application issues.
- Analyze application and container logs.
- Verify ALB, Target Groups and Ingress configuration.
- Check RDS, Kafka, Redis and external dependencies.
- Perform rollback using Jenkins or Kubernetes.
- Prepare RCA and implement preventive measures.

Interviewer Follow-up Questions



1. If Pods are Running but the application is still not accessible, what will you check?

Ans: I will verify the Kubernetes Service, Ingress Controller, ALB Target Group health, DNS resolution, Security Groups and application logs because a Running pod doesn't always mean the application is serving traffic.

2. When will you decide to rollback a deployment?

Ans: If the issue started immediately after deployment and logs confirm application-related failures, I will rollback to the previous stable release to restore production quickly while investigating the root cause separately.

3. How do you identify whether the issue is Infrastructure or Application related?

Ans: I first verify monitoring metrics, infrastructure health, Kubernetes resources, network connectivity and dependencies. If infrastructure is healthy, I analyze application logs and recent code changes to identify the application-level issue.

② Q. Website / Application Not Accessible.

How will you troubleshoot?

Interview Answer :

When users are unable to access the website, I follow an end-to-end approach from outside to inside to identify where the issue is.

First, I confirm whether the issue is for all users or a specific region using tools like DownDetector or by checking with the team.

I start from DNS resolution using `nslookup` / `dig` to ensure the domain is resolving to the correct IP or Load Balancer.

Then I check the ALB / NLB in AWS console. I verify the Target Group health. If targets are unhealthy, the ALB will return 5xx or connection errors.

I verify Listener rules, security groups, and that the correct port (80/443) is open.

If the targets are healthy, I check the Ingress (if on Kubernetes) and Service configuration to ensure traffic is reaching the pods.

Next, I check the application logs for errors, exceptions, or database connectivity issues.

I also verify backend dependencies like RDS, Redis, Kafka, or external APIs because if any dependency is down, the application may not respond.

Finally, I check recent deployments, configuration changes, or certificate expiry which can also break the website.

Once I identify the root cause, I fix the issue, monitor the service and document the findings in RCA.

Troubleshooting Flow (Outside → Inside)



Important Commands



- | | |
|---|-------------------------|
| <code>nslookup <domain></code> | # Check DNS resolution |
| <code>dig <domain></code> | # Detailed DNS lookup |
| <code>curl -I https://<domain></code> | # Check HTTP response |
| <code>aws elbv2 describe-load-balancers</code> | # List ALBs |
| <code>aws elbv2 describe-target-health --target-group-arn <tg-arn></code> | # Target group health |
| <code>kubectl get ingress -A</code> | # List ingresses |
| <code>kubectl get svc -A</code> | # List services |
| <code>kubectl get pods -A</code> | # List pods |
| <code>kubectl logs <pod> -n <namespace></code> | # Pod logs |
| <code>openssl s_client -connect <domain>:443</code> | # SSL certificate check |

Common Causes



- DNS misconfiguration or propagation delay
- ALB / NLB listener or rule misconfiguration
- Target group unhealthy
- Security group / NACL blocking traffic
- Ingress rule not matching the host/path
- Application crash or high error rate
- Database / backend service down
- Recent deployment causing issues
- SSL certificate expired or invalid



Key Responsibilities

- Ensure high availability of web applications.
- Monitor ALB, target groups and instances/pods.
- Configure and manage DNS, SSL, Ingress and Security Groups.
- Troubleshoot network, infra and application issues.
- Collaborate with dev teams for quick resolution.
- Maintain RCA and implement preventive measures.

Interviewer Follow-up Questions



1. If target group is unhealthy, what will you check first?

Ans: I will check instance/pod health, application health checks, security groups, back-end service status and application logs.

2. How do you ensure zero downtime during deployments?

Ans: I use blue-green or rolling deployments with health checks and auto scaling to ensure minimum disruption.

3. How do you monitor website availability?

Ans: I use CloudWatch Alarms, Prometheus, Grafana dashboards and synthetic monitoring to track uptime, latency and errors.



③ Q. SSH Connection Failed.

How will you troubleshoot?

Interview Answer :

When I am unable to SSH into a server, I follow a systematic approach to identify where the connection is failing.

1. First, I check the instance status in AWS Console to make sure the EC2 is running and status checks are 2/2.
2. Then, I verify the Public IP / DNS and try to ping the instance from my local machine or from a jump server.
3. I check the Security Group (Inbound rules) to ensure port 22 is open for my IP or the allowed range.
4. If SG is correct, I check NACL associated with the subnet to ensure inbound/outbound rules allow port 22.
5. If the instance is in a private subnet, I make sure I'm connecting from a Bastion Host or using AWS Session Manager (SSM).
6. I verify the Key Pair I am using is the correct one and the .pem file has proper permissions (chmod 400 key.pem).
7. Then I try SSH with verbose mode to get more details:
`ssh -v -i key.pem ec2-user@<public-ip>`
8. If SSH service is down, I connect via SSM or check the instance logs to see if the OS is working fine.
9. Finally, I document the root cause and take preventive measures like using SSM, limiting IPs in SG and enabling detailed monitoring.

Troubleshooting Flow

- ①  Check EC2 Instance Status Running? Status check 2/2?
- ②  Check Connectivity Ping / Telnet from your machine to Public IP / DNS
- ③  Check Security Group Inbound rule for port 22 open?
- ④  Check NACL Inbound & Outbound allow port 22?
- ⑤  Check Access Method Bastion Host / SSM Session Manager
- ⑥  Check Key Pair & Permissions Correct key? chmod 400 key.pem
- ⑦  Try SSH with Verbose `ssh -v -i key.pem user@ip`
- ⑧  Check SSH Service / Logs Using SSM or console output
- ⑨  Identify Root Cause & Fix Then document & monitor

Important Commands



- `ping <public-ip>` # Check connectivity
- `telnet <public-ip> 22` # Check port 22
- `nc -zv <public-ip> 22` # Check port 22
- `aws ec2 describe-instances --instance-ids i-xxxx` # Check instance status
- `aws ec2 describe-security-groups --group-ids sg-xxxx` # Check SG rules
- `aws ec2 describe-network-acls --network-acl-ids acl-xxxx` # Check NACL rules
- `ssh -v -i key.pem ec2-user@<public-ip>` # Verbose SSH
- `chmod 400 key.pem` # Fix key permission
- `aws ssm start-session --target i-xxxx` # SSM Connect

Common Causes



- Security Group not allowing port 22
- NACL blocking inbound or outbound traffic
- Wrong key pair or wrong username
- Key file permission is open (not 400)
- EC2 instance stopped or in pending state
- Connecting from IP not allowed in SG
- SSH service stopped (sshd not running)
- Trying to SSH to private instance without Bastion / SSM
- Route table / Internet Gateway misconfiguration



Key Responsibilities

- Manage EC2 instances and access securely
- Configure Security Groups, NACLs and Key Pairs
- Use Bastion Host and SSM for secure access
- Automate infra using Terraform
- Monitor instance health with CloudWatch
- Troubleshoot connectivity and network issues

Interviewer Follow-up Questions



1. How do you connect to a private EC2 instance?

Ans: I use a Bastion Host or AWS SSM Session Manager to connect securely without exposing the instance to the internet.

2. What if you lost the key pair and cannot SSH?

Ans: I stop the instance (if possible), detach the root EBS volume, attach it to another instance, update `authorized_keys` or reset the password using EC2Rescue (if enabled).

3. How do you secure SSH access in AWS?

Ans: I allow SSH only from specific IPs, use key-based authentication, use SSM instead of public access, and rotate keys periodically.

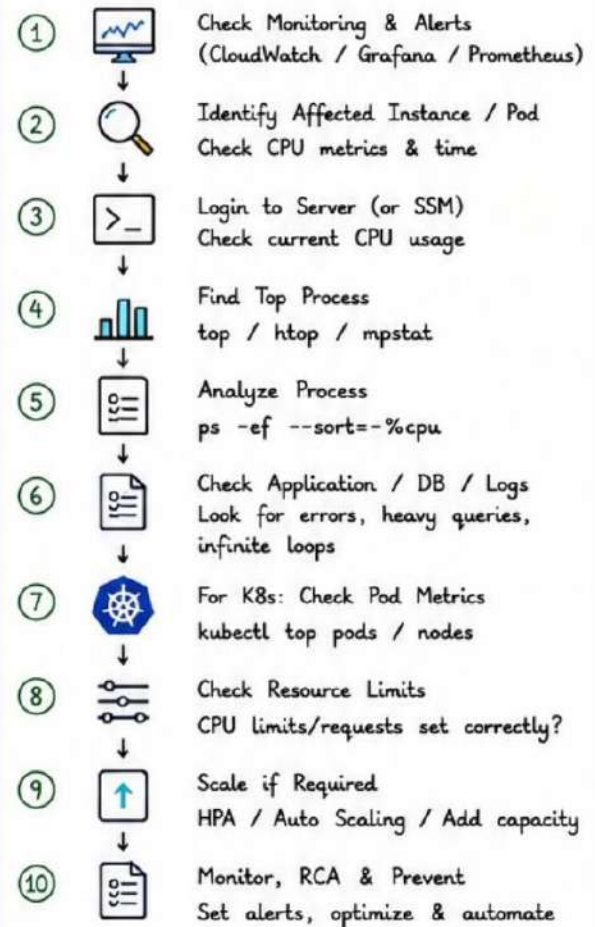
④ Q. High CPU Usage on Server / Instance. How will you troubleshoot?

Interview Answer :

When I see high CPU usage alert, I follow a systematic approach to find the root cause and fix it permanently.

1. First, I check the severity and impact in monitoring tools like CloudWatch, Grafana or Prometheus. I check which instance, service or pod is consuming high CPU and since when.
2. I login to the server (or use SSM) and check the CPU utilization using `top`, `htop` or `mpstat` to see which process is using high CPU.
3. I identify the top consuming process and analyze it. I check the threads, PID and parent process using `ps -ef --sort=-%cpu`.
4. I check if the high CPU is because of application issue, memory pressure (swap), runaway process, cron job, or external load.
5. If it's an application, I check the application logs to see errors, infinite loops, heavy queries or increased traffic.
6. If it's a database, I check slow queries, locks, long running transactions and optimize the queries or add indexes.
7. For containers / Kubernetes, I check the pod metrics using `kubectl top pods / nodes` and identify the pod causing high CPU.
8. I check if resource limits are set correctly. If not, I set proper CPU limits/requests to avoid noisy neighbor issues.
9. If the traffic is genuine, I scale the service (horizontal pod autoscaler or EC2 Auto Scaling) to distribute the load.
10. After resolving the issue, I monitor the instance to ensure CPU usage is stable and then document the RCA and add alerts or automation to prevent the issue in future.

Troubleshooting Flow



Important Commands

- | | |
|---|------------------------------|
| • <code>top</code> | # Real-time processes |
| • <code>htop</code> | # Interactive process viewer |
| • <code>mpstat -P ALL 1</code> | # CPU usage per core |
| • <code>ps -ef --sort=-%cpu head -10</code> | # Top CPU processes |
| • <code>pidstat -u -p <pid> 1</code> | # Stats for specific PID |
| • <code>docker stats</code> | # Check container CPU usage |
| • <code>kubectl top pods -A</code> | # CPU usage of pods |
| • <code>kubectl top nodes</code> | # CPU usage of nodes |
| • <code>journalctl -u <service> --since "1h ago"</code> | # Check logs |
| • <code>aws ec2 describe-instances --instance-ids i-xxxx</code> | # Check CPU info (AWS) |

Common Causes



- Application bugs (infinite loops, memory leaks)
- Heavy DB queries / locks / full table scans
- High traffic without auto scaling
- Cron jobs running too frequently
- Misconfigured containers without CPU limits
- Background processes (backup, sync, antivirus)
- Third-party API timeouts causing retries
- No resource limits in Kubernetes (noisy neighbors)



Key Responsibilities

- Monitor systems using CloudWatch, Grafana, Prometheus and set alerts.
- Troubleshoot performance issues on EC2, EKS and on-prem servers.
- Analyze logs, metrics and application behavior.
- Optimize applications, queries and resources.
- Implement auto scaling and resource limits.
- Perform RCA and implement preventive actions.

Interviewer Follow-up Questions



1. How do you find which process is consuming high CPU?
Ans: I use `top`, `htop` or `ps -ef --sort=-%cpu` to identify the process. Then I analyze the PID, threads and check logs.
2. What if CPU is high because of high traffic?
Ans: I verify metrics and scale the service using HPA (Kubernetes) or Auto Scaling (EC2). I also optimize the app and add caching.
3. How do you prevent high CPU issues in future?
Ans: I set proper alerts, optimize code and queries, configure resource limits, implement auto scaling and perform regular reviews.

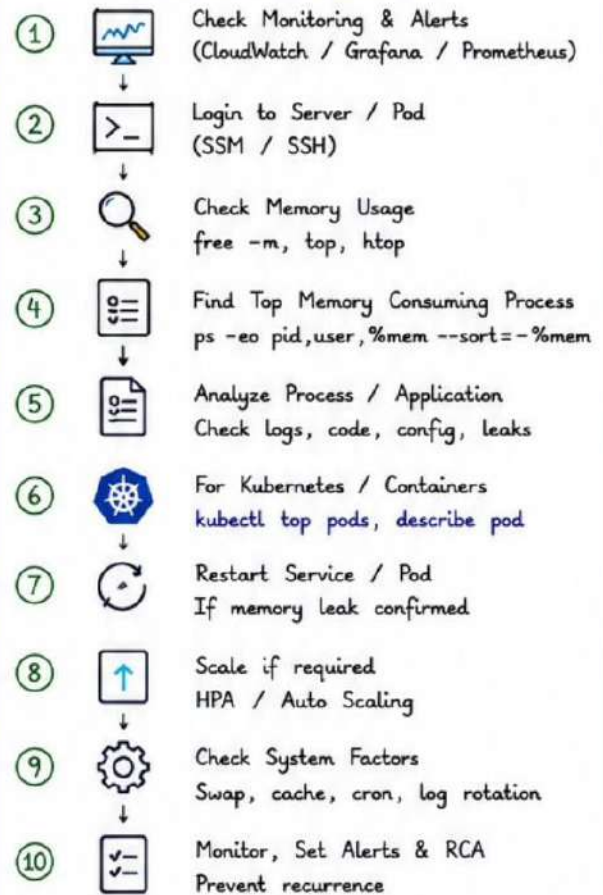
5 Q. High Memory Usage on Server / Instance / Pod. How will you troubleshoot?

Interview Answer :

When I observe high memory usage, I follow a step-by-step approach to find the root cause and resolve it without impacting the application.

1. I first check the memory utilization in CloudWatch / Grafana / Prometheus to confirm the issue and identify the affected instance or pod.
2. I login to the server (or use SSM) and check memory usage using `free -m`, `top` or `htop` command.
3. I identify the top memory consuming process. I check PID, user, Virtual Memory (VSZ), Resident Memory (RSS) using:
`ps -eo pid,user,%mem,vsz,rss,comm --sort=-%mem | head -10`
4. I analyze the process. If it's an application, I check application logs for memory leak, cache issue, or large data in memory.
5. For Java applications, I check JVM heap usage using:
`jstat -gcutil <pid> 5s 5` or `jcmd <pid> GC.heap_info`
6. For containers / Kubernetes, I check pod metrics:
`kubectl top pods -A` and `kubectl describe pod <pod>`
I also check if the container has memory limit set correctly.
7. If a memory leak is suspected, I restart the service or pod gracefully and monitor the memory trend.
8. If it's a temporary spike due to traffic, I scale the service (HPA / Cluster Autoscaler / EC2 Auto Scaling).
9. I check system factors like swap usage, cache usage and background jobs (cron, backup, log rotation).
10. After resolving, I monitor the system, set alerts and document the RCA with preventive steps (optimize code, increase limits, add caching, tune JVM, etc.).

Troubleshooting Flow



Important Commands

- | | |
|--|------------------------------|
| • <code>free -m</code> | # Overall memory usage |
| • <code>top</code> | # Real-time memory usage |
| • <code>htop</code> | # Interactive process viewer |
| • <code>ps -eo pid,user,%mem,vsz,rss,comm --sort=-%mem head -10</code> | # Top memory processes |
| • <code>vmstat 1 5</code> | # Memory stats |
| • <code>cat /proc/meminfo</code> | # Detailed memory info |
| • <code>swapon --show</code> | # Swap usage |
| • <code>jstat -gcutil <pid> 5s 5</code> | # JVM memory usage |
| • <code>jcmd <pid> GC.heap_info</code> | # Heap details |
| • <code>kubectl top pods -A</code> | # Pod memory metrics |
| • <code>kubectl describe pod <pod></code> | # Check limits & events |

Common Causes

- Memory leak in application or libraries
- JVM heap not configured properly
- Large caching (in-memory cache / session store)
- High number of concurrent connections
- Background jobs / cron consuming memory
- Incorrect container memory limits
- Load spike without auto scaling
- Excessive logging / logs in memory
- Not enough system memory / swap



Key Responsibilities

- Monitor memory using CloudWatch, Grafana, Prometheus.
- Troubleshoot application and infrastructure issues.
- Optimize application, JVM and system performance.
- Manage Kubernetes resources and pod limits.
- Implement auto scaling and capacity planning.
- Perform RCA and implement preventative actions.

Interviewer Follow-up Questions



1. How do you identify memory leak in an application?

Ans: I monitor memory trend. If memory keeps increasing and never comes down even after GC, it indicates a memory leak. I check heap dump, logs and code for the root cause.

2. How do you set memory limits for a container in Kubernetes?

Ans: I set resources in the deployment YAML under resources section:

```

resources:
  limits:
    memory: "512Mi"
  requests:
    memory: "256Mi"
  
```

3. What actions do you take if memory is high on all instances?

Ans: I scale out using Auto Scaling / HPA, optimize the application,

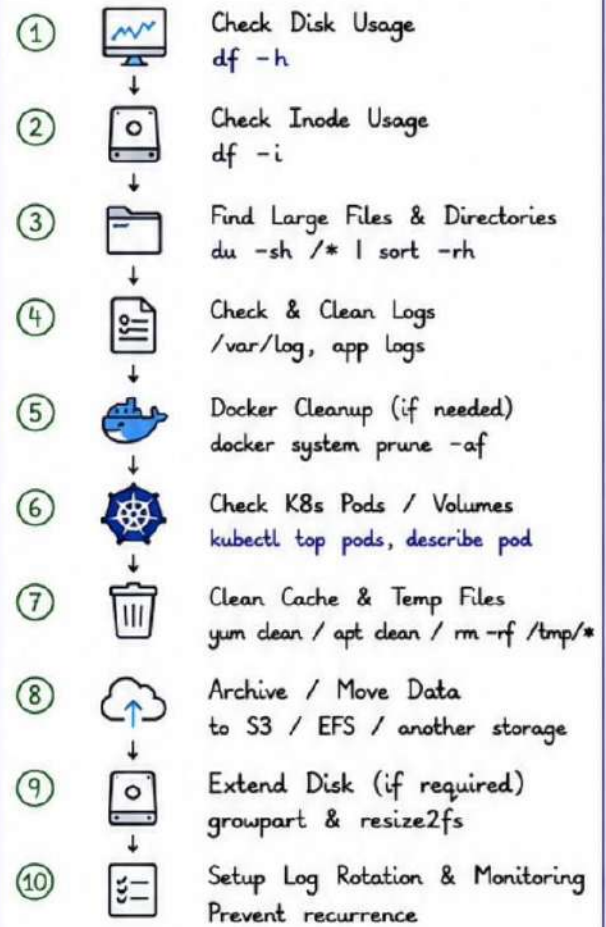
⑥ Q. Disk Full on Server / Instance / Pod. How will you troubleshoot?

Interview Answer :

When I get an alert for disk full issue, I follow a systematic approach to free up space and prevent the issue from recurring.

1. First, I check overall disk usage using `df -h` to identify which filesystem is full.
2. Then I check inode usage using `df -i`, because inode exhaustion can also cause "No space left on device" error.
3. I identify large files and directories using:
`du -sh /* | sort -rh | head -10`
4. I check log directories (`/var/log`, `/opt/logs`, application logs) and remove old or unnecessary logs.
5. If the issue is in `/var/lib/docker`, I run Docker cleanup:
`docker system df -h` `docker system prune -af`
6. For Kubernetes pods, I check `emptyDir` volumes and container logs which may fill the disk:
`kubectl top pods --all-namespaces` (check usage)
`kubectl describe pod <pod>` (check events)
7. I clean package cache and temporary files:
`yum clean all` `apt-get clean` `rm -rf /tmp/*`
8. If the disk is still full, I archive old data or move it to S3/EFS or another storage.
9. If it's an EBS volume and space is still not enough, I extend the volume from AWS Console and run:
`growpart /dev/xvda1` `resize2fs /dev/xvda1`
10. Finally, I set up log rotation and monitoring to prevent the issue from happening again.

Troubleshooting Flow



Important Commands

- | | |
|--|------------------------------|
| <code>df -h</code> | # Check disk usage |
| <code>df -i</code> | # Check inode usage |
| <code>du -sh /* sort -rh</code> | # Find large directories |
| <code>du -sh <dir></code> | # Size of specific directory |
| <code>docker system df -h</code> | # Docker disk usage |
| <code>docker system prune -af</code> | # Clean unused docker data |
| <code>kubectl top pods --all-namespaces</code> | # Check k8s resource usage |
| <code>kubectl describe pod <pod></code> | # Check pod events & errors |
| <code>yum clean all / apt-get clean</code> | # Clean package cache |
| <code>rm -rf /tmp/*</code> | # Clean temp files |
| <code>growpart /dev/xvda1</code> | # Extend EBS partition |
| <code>resize2fs /dev/xvda1</code> | # Resize filesystem |
| <code>aws s3 cp <file> s3://bucket/</code> | # Move/Archive to S3 |

Common Causes

- Large log files and no log rotation
- Docker images / containers consuming space
- Kubernetes `emptyDir` or logs filling disk
- Application generating large files
- Package cache & temporary files
- Database / backup files stored locally
- Insufficient EBS / root volume size
- Inode exhaustion
- Unnecessary files in `/tmp`, `/var`, `/opt`



Key Responsibilities

- Monitor disk usage using CloudWatch & OS tools.
- Manage EBS volumes and storage.
- Implement log rotation & housekeeping.
- Troubleshoot Docker & Kubernetes storage issues.
- Automate cleanup and set alerts.
- Perform RCA and preventive actions.

Interviewer Follow-up Questions



1. How do you prevent disk full issue in future?

Ans: I implement log rotation, set up CloudWatch alarms on disk usage, automate cleanup scripts and review storage regularly.

2. What is the difference between disk full and inode full?

Ans: Disk full means the storage (space) is full. Inode full means all inodes are used, so new files cannot be created even if space is available.

3. How do you extend an EBS volume with zero downtime?

Ans: I increase the volume size in AWS Console, then run `growpart` to expand the partition and `resize2fs` to extend the filesystem without downtime.

7 Q. Docker Container is Down / Exited.

How will you troubleshoot?

Interview Answer :

When I find a docker container is down or exited, I follow a step-by-step approach to identify the root cause and fix it.

1. First, I check the status of all containers:
`docker ps -a`
2. I identify the exited container and check the exit code:
`docker inspect <container_id> | grep ExitCode`
3. I check the logs of the container to understand the error:
`docker logs <container_id> --tail 100`
4. I verify if the container has enough resources (CPU/Memory) and if any limits are hitting.
5. I check the container's healthcheck (if configured):
`docker inspect <container_id> | grep -A 10 Health`
6. I verify if the image is correct and available locally:
`docker images | grep <image_name>`
7. If it's a dependency issue (DB, API, Network), I check connectivity from the container.
8. If the container crashed due to code or config issue, I fix the issue and restart the container.
9. I ensure the restart policy is set correctly (unless-stopped or always) to avoid manual restart.
10. Finally, I monitor the container for some time and set alerts to prevent recurrence.

Troubleshooting Flow

- ①  Check All Containers
`docker ps -a`
- ②  Check Exit Code
`docker inspect | grep ExitCode`
- ③  Check Container Logs
`docker logs <id> --tail 100`
- ④  Check Resource Usage
CPU / Memory / Limits
- ⑤  Check Healthcheck
`docker inspect | grep Health`
- ⑥  Check Image
`docker images | grep <image>`
- ⑦  Check Dependencies & Connectivity
DB / API / Network
- ⑧  Fix Issue & Restart Container
`docker start <id> / restart <id>`
- ⑨  Verify Restart Policy
unless-stopped / always
- ⑩  Monitor & Set Alerts
CloudWatch / Prometheus / Grafana

Important Commands

- `docker ps -a` # List all containers
- `docker ps` # List running containers
- `docker logs <container_id> --tail 100` # Check container logs
- `docker inspect <container_id>` # Inspect container details
- `docker inspect <id> | grep ExitCode` # Check exit code
- `docker inspect <id> | grep -A 10 Health` # Check healthcheck output
- `docker stats` # Check resource usage
- `docker images | grep <image_name>` # Check image locally
- `docker exec -it <id> /bin/bash` # Login to container
- `docker restart <container_id>` # Restart container
- `docker system df -h` # Docker disk usage
- `docker system prune -a` # Clean unused data

Common Causes

- Application crash / code bug
- Wrong configuration / environment variables
- Database / external service not reachable
- Insufficient CPU / Memory
- Healthcheck failing
- Port already in use
- Image not found / corrupted
- Volume / storage issue
- OOMKilled (Out Of Memory)
- Restart policy not set correctly



Key Responsibilities

- Deploy & manage Docker containers in production.
- Monitor containers using CloudWatch, Prometheus.
- Troubleshoot container issues and reduce MTTR.
- Manage images, volumes, networks and resources.
- Automate deployments using Jenkins & Docker.
- Ensure high availability and self-healing.

Interviewer Follow-up Questions



1. How do you check why a container exited?

Ans: I use "docker inspect <id> | grep ExitCode" and check the exit code. Then I check the logs to see the actual error.

2. What is the difference between "docker stop" and "docker kill"?

Ans: docker stop sends SIGTERM and allows the container to shutdown gracefully. docker kill sends SIGKILL and stops immediately.

3. How do you ensure a container automatically restarts?

Ans: I set the restart policy to "unless-stopped" or "always" using --restart unless-stopped while running the container.



Interview Tip

8 Q. Kubernetes Pod is Not Running.

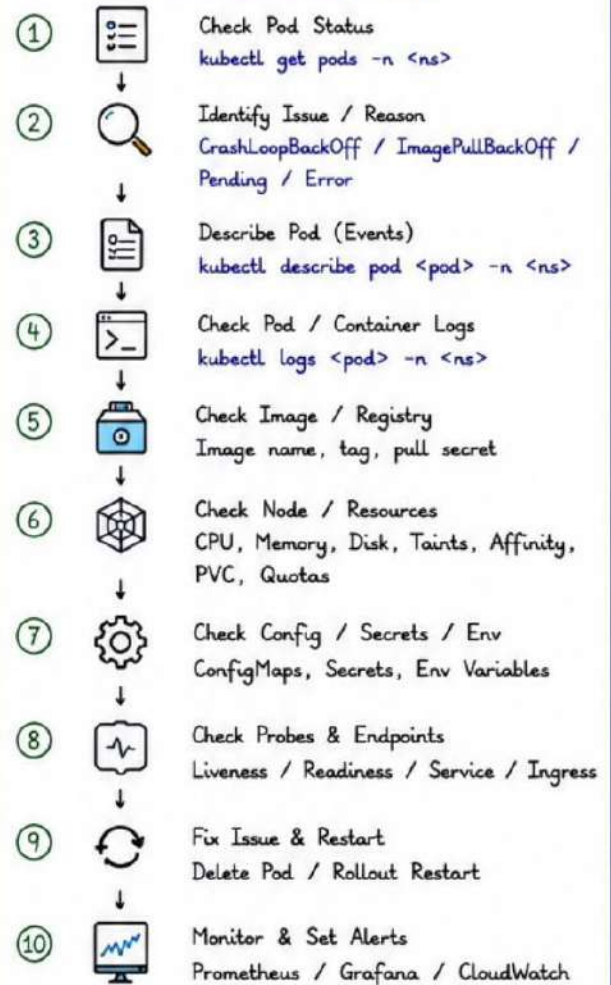
How will you troubleshoot?

Interview Answer :

When I see a Kubernetes pod is not running, I follow a systematic approach to identify the root cause and fix it.

1. First, I check the status of all pods in the namespace:
`kubectl get pods -n <namespace>`
2. I identify the pod which is not running and check its status and reason (CrashLoopBackOff, ImagePullBackOff, Pending, etc.).
3. I describe the pod to get detailed events and reason:
`kubectl describe pod <pod-name> -n <namespace>`
4. I check the container logs to understand the actual error:
`kubectl logs <pod-name> -n <namespace>`
`kubectl logs <pod-name> -n <namespace> -c <container-name>`
5. I check if the issue is due to wrong image, tag or private registry authentication.
6. If the pod is in Pending state, I check node resources, taints, affinity rules, PVC, quotas and insufficient CPU/Memory.
7. I verify ConfigMaps, Secrets, Environment variables and application configuration.
8. I check liveness and readiness probes. If probes are failing, I review the application health and endpoints.
9. After fixing the issue (config, image, resources, etc.), I delete the pod or restart the deployment to create a new pod.
10. Finally, I monitor the pod and set alerts to prevent future occurrences.

Troubleshooting Flow



Important Commands

- | | |
|--|------------------------|
| • <code>kubectl get pods -A</code> | # List all pods |
| • <code>kubectl get pods -n <ns></code> | # Pods in namespace |
| • <code>kubectl describe pod <pod> -n <ns></code> | # Pod details & events |
| • <code>kubectl logs <pod> -n <ns></code> | # Pod logs |
| • <code>kubectl logs <pod> -c <container></code> | # Container logs |
| • <code>kubectl get events -n <ns> --sort-by=.lastTimestamp</code> | # Recent events |
| • <code>kubectl get nodes</code> | # Node status |
| • <code>kubectl top nodes</code> | # Node resource usage |
| • <code>kubectl top pods -n <ns></code> | # Pod resource usage |
| • <code>kubectl describe node <node></code> | # Node details |
| • <code>kubectl rollout restart deployment/<deploy> -n <ns></code> | # Restart deployment |

Common Causes

- Wrong / unavailable Docker image or tag
- Private registry image pull failure (missing secret)
- Insufficient CPU/Memory on node
- Liveness / Readiness probe failing
- Application crashes / runtime error
- ConfigMap / Secret / Environment issue
- PVC not bound / storage issue
- Node taints / affinity / anti-affinity rules
- Network / DNS / Service connectivity issue
- Resource quota / limit range violation



Key Responsibilities

- Deploy and manage applications on EKS.
- Monitor pod health and fix issues proactively.
- Troubleshoot pods, services, ingress and workloads.
- Manage ConfigMaps, Secrets, PVCs and RBAC.
- Implement resource limits and auto-scaling.
- Monitor using Prometheus, Grafana and CloudWatch.
- Reduce MTTR and ensure high availability.

Interviewer Follow-up Questions

1. What is the difference between CrashLoopBackOff and ImagePullBackOff?

Ans: CrashLoopBackOff means the container starts but crashes repeatedly. ImagePullBackOff means Kubernetes is unable to pull the image.

2. Why is a pod in Pending state?

Ans: Due to insufficient resources, node taints, affinity rules, unbound PVC, image pull issue or quota limits.

3. How do you check why a pod is not scheduled on any node?

Ans: I check events using `'kubectl describe pod <pod>'` and look for reasons like taints, insufficient resources or PVC issues.



9 Q. Database Connection Issues in Application. How will you troubleshoot?

Interview Answer :

When I see application errors related to database connection, I follow a systematic approach to identify the root cause and fix it.

1. I first check the application logs to understand the exact error (timeout, authentication failed, too many connections, etc.).
2. I verify if the database service is running and healthy.
`mysql -u root -p -h <host> -P <port>` or `pg-isready -h <host>`
3. I check network connectivity from the application server to the database server.
`telnet <db-host> <port>` or `nc -zv <db-host> <port>`
4. I validate the database credentials and permissions.
login using same user from CLI and test connection.
5. I check if the database is overloaded.
Check CPU, Memory, Disk, Connections.
`SHOW PROCESSLIST;` or `SELECT count(*) FROM pg_stat_activity;`
6. I check for too many connections.
`SHOW VARIABLES LIKE 'max_connections';`
And current usage from process list.
7. I review slow queries and locks which may be blocking connections.
8. I check firewall, security group rules and database port accessibility.
9. If it is a RDS/Aurora issue, I check CloudWatch metrics, events, and RDS status.
10. After identifying the issue, I fix it (kill idle connections, increase `max_connections`, fix network, update credentials, etc.).
11. Finally, I monitor the application and database to ensure the issue is resolved and will not reoccur.

Troubleshooting Flow

- 1 Check Application Logs
Identify DB connection error
- 2 Check DB Service Status
`mysql / pg-isready`
- 3 Check Network Connectivity
`telnet / nc`
- 4 Validate Credentials & Permissions
Test login from CLI
- 5 Check DB Resource Usage
CPU / Memory / Disk / Connections
- 6 Check Too Many Connections
`SHOW PROCESSLIST;`
`max_connections`
- 7 Check Slow Queries & Locks
Review slow logs & locks
- 8 Check Firewall / SG Rules
Verify port accessibility
- 9 For RDS / Aurora
Check CloudWatch metrics / events
- 10 Fix Issue
Kill idle connections, update config,
fix network, increase limits
- 11 Monitor & Prevent Recurrence
Set alerts, optimize, document RCA

Important Commands

- | | |
|---|-------------------------------|
| <code>mysql -u root -p</code> | # Connect to MySQL |
| <code>pg-isready -h <host> -p <port></code> | # Check Postgres service |
| <code>telnet <host> <port></code> | # Check port connectivity |
| <code>nc -zv <host> <port></code> | # Alternative to telnet |
| <code>SHOW PROCESSLIST;</code> | # MySQL active connections |
| <code>SHOW VARIABLES LIKE 'max_connections';</code> | # MySQL max connections |
| <code>SELECT count(*) FROM pg_stat_activity;</code> | # Postgres active connections |
| <code>SHOW FULL PROCESSLIST;</code> | # MySQL detailed list |
| <code>SHOW ENGINE INNODB STATUS\G</code> | # InnoDB status |
| <code>netstat -tulpn grep <port></code> | # Check port usage |
| <code>lsof -i :<port></code> | # Process using the port |

Common Causes

- Wrong username / password
- Database service is down
- Network connectivity / firewall issues
- Database server overloaded
- Too many connections (exceeding limit)
- Long running queries / locks
- Idle connections not closed
- Incorrect connection string / hostname / port
- SSL / TLS configuration mismatch
- Storage / Disk full on DB server
- Resource limits on RDS (max connections, CPU)



Key Responsibilities

- Monitor application and database health.
- Troubleshoot DB connection and performance issues.
- Manage database users, permissions and access.
- Optimize queries and database performance.
- Manage connection pooling and limits.
- Monitor RDS / Aurora using CloudWatch.
- Implement alerting and preventive measures.
- Perform RCA and document solutions.

Interviewer Follow-up Questions

1. How do you check the number of active connections in MySQL?

Ans: I use "SHOW PROCESSLIST;" or "SHOW STATUS LIKE 'Threads_connected';" to check current active connections.

2. What is the difference between `max_connections` and `Threads_connected`?

Ans: `max_connections` is the maximum allowed connections. `Threads_connected` is the current number of active connections.

3. How do you handle too many connections issue?

Ans: I kill idle connections, increase `max_connections` (if required), optimize the queries, implement connection pooling and scale the DB if needed.

4. What will you do if the DB is slow?

Ans: I check slow query logs, analyze execution plans, kill long running queries, add indexes and optimize the database.



10 Q. Jenkins Build Failed.

How will you troubleshoot?

Interview Answer :

When a Jenkins build fails, I follow a step-by-step approach to identify the failure stage and resolve the issue quickly.

1. I check the build console output and identify the exact stage where the build failed.
2. I review the error message and logs to understand the root cause.
3. I check if the failure is due to code, dependencies, configuration, plugins, environment or infrastructure.
4. I verify the source code changes (latest commit, merge conflicts, syntax errors).
5. I re-run the failed stage or build with `--stacktrace` for Maven/Gradle for more details.
6. I check the workspace, artifacts and test reports for more clues.
7. I verify the build tools and dependencies (Maven/Gradle, npm, pip, Docker image, etc.).
8. I check disk space, memory and CPU usage on the Jenkins server.
`df -h | free -m | top`
9. I verify environment variables, credentials, and secrets used in the pipeline.
10. I check external systems like Nexus, Docker registry, Git repo, SonarQube, etc. for connectivity issues.
11. If it's a flaky issue, I re-run the build to confirm.
12. After fixing, I re-trigger the build and monitor the result.
13. Finally, I document the issue, resolution and suggest improvements to prevent future failures.

Troubleshooting Flow

- ①  Check Build Console Output
Identify failed stage
- ②  Review Error Message & Logs
Understand root cause
- ③  Identify Failure Type
Code / Config / Plugin / Infra
- ④  Check Source Code Changes
Latest commit, conflicts
- ⑤  Re-run with `--stacktrace`
(Maven/Gradle)
- ⑥  Check Workspace, Artifacts
& Test Reports
- ⑦  Verify Tools & Dependencies
Maven/Gradle, npm, Docker
- ⑧  Check Server Resources
Disk / Memory / CPU
- ⑨  Verify Env Variables, Credentials
& Secrets
- ⑩  Check External Systems
Nexus, Git, Docker, SonarQube
- ⑪  Re-run Build to Confirm
Flaky or Intermittent Issues
- ⑫  Fix, Re-trigger, Monitor & Document
Prevent future failures

Important Commands

- | | |
|--|-----------------------|
| • <code>jenkins-cli.jar -s <url> list-jobs</code> | # List all jobs |
| • <code>jenkins-cli.jar -s <url> build <job></code> | # Trigger job |
| • <code>jenkins-cli.jar -s <url> console <job> <build#></code> | # View console logs |
| • <code>jenkins-cli.jar -s <url> get-job <job> grep logrotate</code> | Check logrotate |
| • <code>df -h</code> | # Disk space usage |
| • <code>free -m</code> | # Memory usage |
| • <code>top</code> | # CPU usage |
| • <code>tail -f /var/log/jenkins/jenkins.log</code> | # Jenkins master log |
| • <code>tail -f /var/log/messages</code> | # System logs |
| • <code>docker system df</code> | # Docker space usage |
| • <code>docker ps -a</code> | # List all containers |
| • <code>docker logs <container_id></code> | # Container logs |
| • <code>mvn -X clean install</code> | # Maven debug build |
| • <code>npm install --verbose</code> | # NPM verbose build |

Common Causes

- Code compilation errors / test case failures
- Dependency download failure (Nexus / Internet)
- Plugin incompatibility or outdated plugins
- Out of memory on Jenkins or build agent
- Disk full on Jenkins server or agent
- Wrong credentials or expired tokens
- Network connectivity issues
- Incorrect pipeline script / syntax
- Docker build / pull / push failures
- External tool failures (SonarQube, Nexus, Git, etc.)
- Flaky / intermittent infrastructure issues

Key Responsibilities

- Maintain Jenkins CI/CD pipelines.
- Troubleshoot build, deployment and integration failures.
- Manage Jenkins plugins, credentials and nodes.
- Monitor build performance and success rate.
- Optimize pipelines and reduce build time.
- Ensure high availability of Jenkins.
- Implement alerts and notifications.
- Document issues and share knowledge.

Interviewer Follow-up Questions

1. How do you find the root cause of a failed build?

Ans: I check the console output, error logs, failed stage, stacktrace and related artifacts. Then I identify if it's code, config, dependency or infra issue.

2. What is the use of `--stacktrace` in Maven/Gradle?

Ans: It provides detailed stacktrace and debug information which helps in identifying the exact error and root cause.

3. How do you handle flaky builds?

Ans: I re-run the build, check for network or resource issues, add retry mechanism in the pipeline and stabilize the environment.

4. How do you reduce Jenkins build time?

Ans: I use parallel stages, caching, optimized Docker images, artifact reuse, clean workspace, and incremental builds.

11 Q. AWS EC2 Instance Troubleshooting.

How will you troubleshoot?

Interview Answer :

When an EC2 instance is not working as expected, I follow a systematic approach to identify the root cause and fix it.

1. First, I check the instance state in the AWS Console.
If it's stopped, terminated or in a warning state, I take appropriate action.
2. I check the instance status checks (System & Instance) to see if the instance is healthy.
3. I connect to the instance using SSH (for Linux) or RDP (for Windows) and verify the basic connectivity.
4. I check the logs:
 - Linux : `/var/log/messages`, `/var/log/cloud-init-output.log`
 - Windows : Event Viewer
5. I verify the security group and NACL rules to ensure required ports are open.
6. I check the attached EBS volumes and their status.
7. I check the instance resource utilization (CPU, Memory, Disk) using CloudWatch.
8. If the instance is not reachable, I try to reboot it from the Console.
9. If still not working, I create a snapshot of the instance (if possible) and launch a new instance.
10. Finally, I document the issue, root cause and the fix for future reference.

Troubleshooting Flow



Important Commands

- `aws ec2 describe-instances`
- `aws ec2 describe-instance-status`
- `aws ec2 reboot-instances --instance-ids <id>`
- `ssh -i key.pem ec2-user@<public-ip>`
- `aws ec2 describe-volumes`
- `aws ec2 create-snapshot --volume-id <vol-id>`
- `df -h`
- `free -m`
- `top / htop`
- `tail -f /var/log/messages`
- Get-WinEvent (Windows Event Viewer)

Common Causes

- Instance not running (stopped/terminated)
- Security group / NACL blocking access
- Incorrect SSH key / permissions
- EBS volume detached or in error
- High CPU / Memory usage
- Instance status check failed
- Incorrect IAM permissions
- User data script failure
- OS / application crash
- Network connectivity issues

Key Notes

- Always check the basics first
- Use AWS Console + CLI together
- Logs are your best friend
- Take snapshots before making changes
- Monitor with CloudWatch
- Document and communicate clearly



Interview Tip

Always demonstrate a clear step-by-step troubleshooting process. Mention tools, commands and AWS services you will use.

Interviewer Follow-up Questions

1. What is the first thing you check when an EC2 instance is not reachable?
Ans: Check instance state and status checks in AWS Console.

2. How do you check the logs of an EC2 instance?
Ans: SSH into the instance and check `/var/log/messages` or use CloudWatch.

3. How do you recover a failed EC2 instance?
Ans: Reboot the instance. If still not working create snapshot and launch new instance.



12 Q. Service / Application is Not Restarting. How will you troubleshoot?

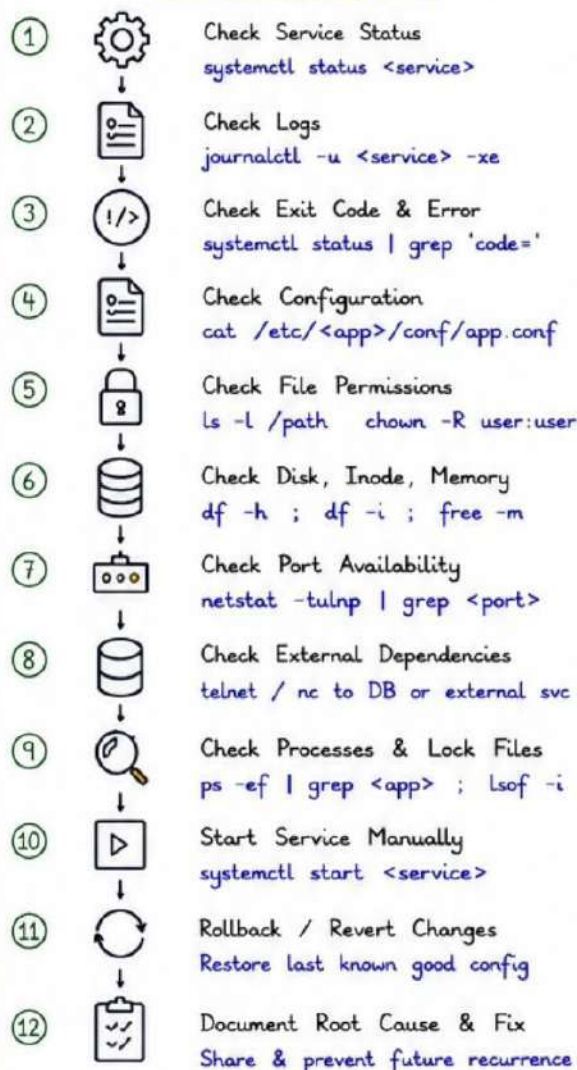
Experience : 5 Years

Interview Answer :

When a service or application is not restarting, I follow a systematic approach to identify the root cause and fix it.

1. I check the service status.
Linux : `systemctl status <service-name>`
Windows : `Get-Service <service-name>`
2. I check the recent logs to understand the failure reason.
Linux : `journalctl -u <service-name> -xe`
or `/var/log/<app-name>.log`
Windows : Event Viewer → Application / System Logs
3. I verify the exit code and error message.
`systemctl status <service-name> | grep 'code='`
4. I check the configuration file for any syntax or invalid changes.
`cat /etc/<app>/conf/app.conf`
5. I verify file permissions and ownership of required files/directories.
`ls -l /path/to/app ; chown -R user:user /path`
6. I check disk space, inode usage and memory.
`df -h ; df -i ; free -m`
7. I check if required ports are already in use.
`netstat -tulnp | grep <port>` or `ss -tulnp`
8. I verify database / external dependency connectivity.
`telnet <host> <port>` or `nc -zv <host> <port>`
9. I check if there are any background jobs or processes holding the port or lock files.
`ps -ef | grep <app>` ; `lsof -i :<port>`
10. I try to start the service manually and check the error.
`systemctl start <service-name>`
11. If the issue persists, I rollback the recent changes or revert to last known good configuration.
12. Finally, I document the issue, root cause and resolution for future reference.

Troubleshooting Flow



Important Commands

<code>systemctl status <service-name></code>	# Check status
<code>systemctl start <service-name></code>	# Start service
<code>systemctl restart <service-name></code>	# Restart service
<code>systemctl enable <service-name></code>	# Enable on boot
<code>journalctl -u <service-name> -xe</code>	# View logs
<code>cat /var/log/<app>.log</code>	# App logs
<code>df -h ; df -i</code>	# Disk & inode usage
<code>free -m</code>	# Memory usage
<code>netstat -tulnp grep <port></code>	# Check port
<code>ss -tulnp</code>	# Alternative
<code>ps -ef grep <app></code>	# Check process
<code>lsof -i :<port></code>	# Port in use
<code>chown -R user:user /path</code>	# Change owner
<code>chmod -R 755 /path</code>	# Change permission
<code>telnet <host> <port></code>	# Check connectivity
<code>nc -zv <host> <port></code>	# Netcat check

Common Causes

- Incorrect configuration / syntax error
- Missing or wrong file permissions
- Insufficient disk space or inode
- Port already in use
- Database / external service down
- Wrong service user / permissions
- Out of memory
- Corrupted application files
- Lock files / zombie processes
- Recent changes / deployment issue
- Dependency or environment variables issue
- Firewall / SELinux / AppArmor blocking

Best Practices

- Always check logs first.
- Keep configurations in version control.
- Monitor disk, memory and processes.
- Use proper alerts & notifications.
- Validate changes in lower environment.
- Maintain rollback plan.
- Document everything for knowledge sharing.
- Automate health checks and restarts where applicable.



Key Responsibilities

Ensure services / applications are available & stable.

- Monitor and troubleshoot production issues.
- Reduce MTTR and prevent recurrence.
- Maintain system health and performance.
- Document issues and resolutions.

Interviewer Follow-up Questions



1. What is the first thing you check when a service is not restarting?
Ans: I check the service status and recent logs to identify the immediate error.
2. How do you find out what is using a specific port?
Ans: I use `netstat -tulnp | grep <port>` or `ss -tulnp` to identify the process.
3. What if the service keeps crashing repeatedly?
Ans: I check logs, resource usage, dependencies, recent changes and rollback if needed.
4. How do you handle configuration changes safely?

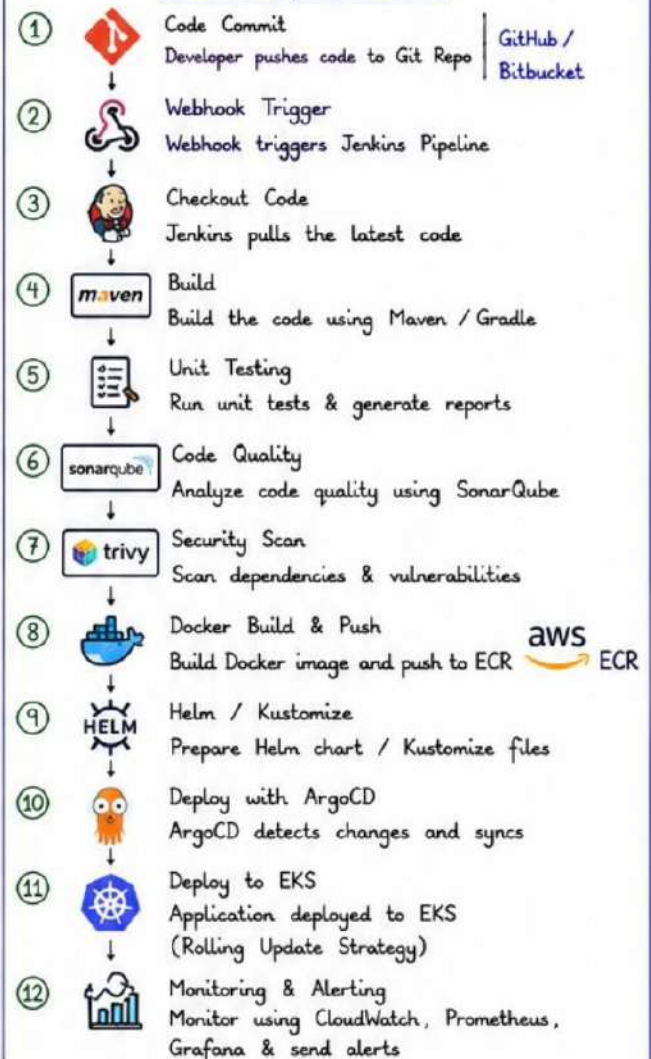
13 Q. Explain your complete CI/CD pipeline from code commit to production deployment.

Interview Answer :

I follow a fully automated CI/CD pipeline to build, test, scan and deploy applications to production with security and reliability.

1. Developer pushes code to Git (GitHub / Bitbucket).
2. Webhook triggers Jenkins pipeline automatically.
3. Jenkins pulls the latest code from repository.
4. Code compilation & build using Maven / Gradle.
5. Unit tests are executed and test reports are generated.
6. Code quality check using SonarQube.
7. Dependency & vulnerability scan using Trivy / OWASP.
8. Docker image is built and pushed to Amazon ECR.
9. Helm chart / Kustomize is prepared for deployment.
10. ArgoCD (GitOps) detects the changes and syncs with Kubernetes (EKS).
11. Application is deployed to EKS with rolling update strategy (zero downtime).
12. Post deployment validation, monitoring and alerting using CloudWatch, Prometheus & Grafana.

CI/CD Pipeline Flow



Important Commands

- | | |
|---|------------------------|
| • git clone <repo-url> | # Clone repository |
| • git add . | # Add changes |
| • git commit -m "msg" | # Commit changes |
| • git push origin main | # Push code |
| • mvn clean install | # Build with Maven |
| • mvn test | # Run unit tests |
| • sonar-scanner | # SonarQube scan |
| • trivy fs | # Scan filesystem |
| • docker build -t <image> . | # Build Docker image |
| • docker tag <image> <ecr-url> | # Tag image |
| • docker push <ecr-url> | # Push to ECR |
| • helm upgrade --install <app> ./chart | # Deploy using Helm |
| • kubectl get pods -n <ns> | # Check pods |
| • kubectl rollout status deploy/<app> -n <ns> | # Check rollout status |
| • kubectl rollback deploy/<app> -n <ns> | # Rollback deployment |

Tools & Technologies Used

- Version Control : Git, GitHub / Bitbucket
- CI Tool : Jenkins
- Build Tool : Maven / Gradle
- Code Quality : SonarQube
- Security Scan : Trivy, OWASP Dependency Check
- Containerization : Docker
- Container Registry : Amazon ECR
- Deployment : ArgoCD (GitOps) / Helm
- Orchestration : Kubernetes (EKS)
- Monitoring : CloudWatch, Prometheus, Grafana
- Notification : SNS, Slack, Email



Key Responsibilities

- Design and maintain CI/CD pipelines.
- Automate build, test and deployment.
- Ensure code quality and security.
- Manage container images and registries.
- Deploy applications on Kubernetes (EKS).
- Monitor applications and infrastructure.
- Implement alerts and dashboards.
- Ensure high availability and zero downtime.
- Optimize pipeline performance.
- Troubleshoot pipeline & deployment issues.

Interviewer Follow-up Questions



1. Why do you use Jenkins for CI/CD?

Ans: Jenkins is highly customizable, has a large plugin ecosystem and is widely used for building complex pipelines.

2. Why do you push Docker images to ECR?

Ans: ECR is secure, scalable and integrates natively with AWS services like EKS.

3. How do you ensure zero downtime during deployment?

Ans: I use rolling update strategy in Kubernetes and readiness probes to ensure only healthy pods receive traffic.

4. What is ArgoCD and how is it useful?

Ans: ArgoCD is a GitOps continuous delivery tool that ensures desired state in Kubernetes matches the declared state in Git.

14 Q. How do you deploy applications in Kubernetes with zero downtime?

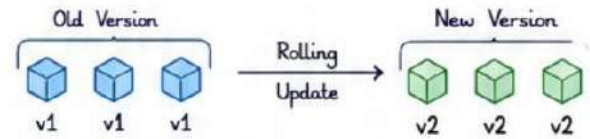
Interview Answer :

To achieve zero downtime, I use different deployment strategies in Kubernetes like Rolling Update, Blue-Green Deployment or Canary Deployment based on the requirement. I also use readiness/liveness probes, HPA and proper monitoring to ensure application availability.

1. I package the application as a Docker image and push it to Amazon ECR.
2. I update the image tag in the Kubernetes manifest (Deployment).
3. I use Rolling Update strategy (default) to deploy the new version gradually.
4. Kubernetes creates new pods with the new version.
5. Readiness probe ensures the new pod is healthy before receiving traffic.
6. Once the new pod is ready, traffic is shifted from old pod to new pod.
7. Old pods are terminated gracefully.
8. If any issue occurs, I rollback to the previous stable version instantly.
9. I use HPA to handle traffic spikes automatically.
10. I monitor the application using Prometheus + Grafana and set alerts in CloudWatch.
11. I store configurations in ConfigMap and sensitive data in Secrets.
12. I use Ingress Controller for traffic routing and TLS termination.

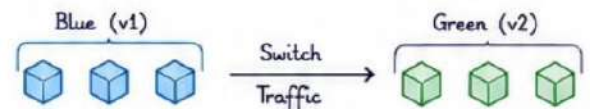
Deployment Strategies in Kubernetes

① Rolling Update (Default)



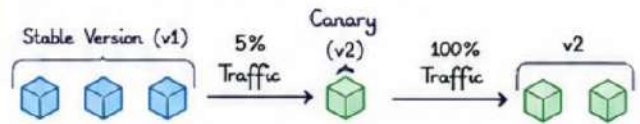
- New pods are created one by one.
- Traffic is shifted gradually.
- Old pods are terminated after new pod is ready.
- Zero downtime.

② Blue-Green Deployment



- Two environments (Blue & Green).
- Deploy new version in Green.
- Test & validate.
- Switch traffic from Blue to Green.
- Zero downtime & easy rollback.

③ Canary Deployment



- Deploy new version to small % of users.
- Monitor metrics & logs.
- Gradually increase traffic.
- Full rollout if everything is fine.
- Zero downtime with less risk.

Key Kubernetes Concepts Used

- **Deployment** : Manages replica set and update strategy
- **Readiness Probe** : Ensures pod is ready before receiving traffic
- **Liveness Probe** : Ensures pod is running, restarts if failed
- **HPA** : Auto scale pods based on CPU/Memory
- **ConfigMap** : Store non-sensitive configuration
- **Secret** : Store sensitive data (password, keys, tokens)
- **Ingress** : Manage external access & routing
- **Service** : Stable networking for pods
- **PodDisruptionBudget (PDB)** : Ensure min pods available

Important Commands

- | | |
|--|------------------------|
| • <code>kubectl get pods -o wide</code> | # List pods |
| • <code>kubectl get deployments</code> | # List deployments |
| • <code>kubectl describe pod <pod-name></code> | # Describe pod |
| • <code>kubectl describe deploy <deploy-name></code> | # Describe deployment |
| • <code>kubectl rollout status deploy/<name></code> | # Check rollout status |
| • <code>kubectl rollout history deploy/<name></code> | # Rollout history |
| • <code>kubectl rollout undo deploy/<name></code> | # Rollback |
| • <code>kubectl get hpa</code> | # List HPA |
| • <code>kubectl get ingress</code> | # List Ingress |
| • <code>kubectl get svc</code> | # List Services |
| • <code>kubectl logs <pod-name></code> | # View logs |
| • <code>kubectl exec -it <pod> -- sh</code> | # Access pod |



Best Practices

- Always use Rolling Update or Blue-Green for zero downtime.
- Configure readiness & liveness probes properly.
- Set resource requests & limits for all containers.
- Use HPA to handle traffic spikes.
- Monitor using Prometheus & Grafana.
- Keep your Docker images small and secure.
- Automate deployment using CI/CD pipelines.
- Always test in lower environment before production.

15 Q. A P1 Production Incident has occurred.

Explain your end-to-end approach.

Interview Answer :

When a P1 incident occurs in production, I follow a structured approach to minimize the impact, restore service and prevent recurrence. My approach includes the following steps :

1. **Alert Received** : I receive alert from monitoring tools (CloudWatch / Grafana / Prometheus / PagerDuty).
2. **Acknowledge & Triage** : Acknowledge the alert immediately and validate the severity and impact.
3. **War Room / Bridge Call** : Create war room call and inform relevant stakeholders.
4. **Gather Information** : Collect all required details - error logs, metrics, timestamps, recent deployments, changes.
5. **Identify the Impact** : Identify which services are impacted and the business impact (users affected, transactions failing).
6. **Initial Investigation** : Check dashboards, logs, metrics, traces and infrastructure health.
7. **Root Cause Identification** : Identify the root cause of the issue (code, config, infra, dependency, external service etc.).
8. **Implement Fix (Temporary)** : Apply immediate fix or workaround to restore service.
9. **Validate & Monitor** : Validate if the issue is resolved and closely monitor the application.
10. **Permanent Fix** : Develop and deploy permanent fix through proper CI/CD pipeline.
11. **RCA (Root Cause Analysis)** : Perform RCA and document what happened, why it happened and how it was resolved.
12. **Preventive Actions** : Implement preventive measures to avoid similar incidents in future.

P1 Incident Handling Flow

- ①  Alert Received
Monitoring tools trigger P1 alert
- ②  Acknowledge & Triage
Acknowledge and validate the issue
- ③  War Room / Bridge Call
Inform team & relevant stakeholders
- ④  Gather Information
Collect logs, metrics, recent changes
- ⑤  Identify the Impact
Identify affected services & users
- ⑥  Initial Investigation
Check dashboards, logs, metrics
- ⑦  Root Cause Identification
Find the root cause of the issue
- ⑧  Implement Fix (Temporary)
Apply immediate fix / workaround
- ⑨  Validate & Monitor
Validate the fix & monitor closely
- ⑩  Permanent Fix
Code fix & deploy via CI/CD
- ⑪  RCA (Root Cause Analysis) :
Document what happened & why
- ⑫  Preventive Actions
Implement measures to prevent recurrence.

Important Commands / Tools

- | | |
|---|------------------------|
| • kubectl get pods -A | # List all pods |
| • kubectl describe pod <pod-name> | # Describe pod |
| • kubectl logs <pod-name> -f | # View logs |
| • kubectl top pods | # Resource usage |
| • aws logs tail /aws/ecc/<log-group> --follow | # CloudWatch logs |
| • df -h | # Disk usage |
| • free -m | # Memory usage |
| • netstat -tulnp | # Check open ports |
| • systemctl status <service> | # Service status |
| • journalctl -u <service> -xe | # Service logs |
| • grep -i "error" <log-file> | # Search error in logs |

Common Root Causes



- Code bug / recent bad deployment
- Incorrect configuration change
- Database / external dependency down
- High CPU / Memory / Disk usage
- Insufficient resources
- Network connectivity issues
- Expired certificates / secrets
- Third party service failure
- Unhandled exceptions
- Queue backlog / timeout issues

Best Practices



- Have proper alerting & monitoring in place.
- Keep dashboards & runbooks ready.
- Automate rollback wherever possible.
- Communicate clearly & frequently.
- Document everything during incident.
- Perform RCA without blaming.
- Implement preventive & corrective actions.
- Test recovery & failover regularly.



Key Responsibilities

- Monitor production systems 24x7.
- Respond to P1 / P2 incidents.
- Troubleshoot applications & infrastructure.
- Coordinate with dev, ops, db, network teams.
- Minimize MTTR and business impact.
- Perform RCA and implement fixes.
- Maintain runbooks and incident docs.

Interviewer Follow-up Questions



1. How do you decide the severity of an incident?

Ans: Based on the business impact, number of users affected, service availability and SLA / SLO breach.

2. What tools do you use for monitoring and alerting?

Ans: CloudWatch, Prometheus, Grafana, PagerDuty, Datadog, New Relic.

3. How do you communicate during an incident?

Ans: I create a war room call, share regular updates, use Slack / Teams channels and keep stakeholders informed.

4. How do you prevent similar incidents in future?



Interview Tip